



PARKING SYSTEM

OBJECT-ORIENTED PROGRAMMING



Done by:

Noor Al Rawahi (24-0680)

Mazayin Al Busaidi (24-0572)

Rayyan Al Abdul Salam (24-0413)

Shatha Al Busaidi (24-0143)

Basaier Al Kharusi (24-0071)

DESCRIPTION OF THE SYSTEM



Purpose

The purpose of this system is to manage parking slots in an organized and efficient way. It helps users reserve parking spaces, park vehicles, track parking duration, and calculate payment based on slot type and time.

Main Features

- Register customer through app/website
- Show all parking slots
- Reserve a parking slot before arrival
- Verify reservation at the gate
- Track parking duration
- Calculate payment
- Detect if a customer parks in a different slot from the reservation
- Support sunny, shaded, and disabled-only slots

LIST OF MAIN CLASSES

- | | |
|----------------|----------------------|
| 1. Customer | 5. Reservation |
| 2. ParkingSlot | 6. ParkingSession |
| 3. SunnySlot | 7. payment |
| 4. ShadedSlot | 8. ParkingGateSystem |



RESPONSIBILITY OF EACH CLASS

Customer

Stores customer information and checks that user inputs are valid.

ParkingSlot

Base class that represents a parking space

SunnySlot

A type of parking slot with a lower hourly rate.

ShadedSlot

A type of parking slot with a higher hourly rate.

Reservation

Stores information about customer's parking slot reservation

ParkingSession

Tracks parking time and duration

Payment

Calculates cost based on time and violations

ParkingGateSystem

Controls the whole system (entry, parking, payment)



OOP CONCEPTS



Inheritance

- SunnySlot and ShadedSlot inherits from ParkingSlot.

```
class SunnySlot final : public ParkingSlot {...}
```

Composition

- ParkingGateSystem owns slots, reservations, sessions, payments

```
Reservation reservations[MAX];
```

Aggregation

- ParkingGateSystem uses Customer objects, which exist independently

```
Vector<ParkingSlot*> slots;
```

Polymorphism

- getType() and getRate() are virtual overridden in each subclass

```
virtual string getType() const = 0;  
virtual double getRate() const = 0;
```

Encapsulation

- All class data is private; accessed only through public methods

```
private: int customerID;  
public: int getID() const;
```

Operator Overload

- == operator overloaded in Customer and ParkingSlot

```
bool operator==(const Customer& other)  
{return customerID == other.customerID;}
```


SYSTEM DEMONSTRATION

1

Register Customer

ID (8 digits) ·
Name · Phone
(Oman 8-digit) ·
Plate · Disabled
permit verification

2

Show Parking Slots

Displays all 6
slots with type,
disabled flag, and
current status
(Available /
Reserved /
Occupied)

3

Reserve a Slot

Customer selects
a slot → system
validates
availability +
disabled-only
rules → issues
Reservation ID

4

Enter Gate & Park

Driver enters
Reservation ID +
actual slot →
system records
Oman time
automatically →
detects wrong-
slot violations

5

Exit Gate & Pay

System reads
current Oman
time → calculates
duration →
applies rate +
penalties →
displays total in
OMR → frees slot

Menu options 0–5 · Input validation on every field · Real-time Oman clock via <ctime>

WORK DISTRIBUTION

Noor Al Rawahi (24-0680) → Customer Class

Responsible for customer information management, input validation, and disabled permit verification.

Rayyan Al Abdul Salam (24-0413) → ParkingSlot Class, SunnySlot & ShadedSlot

Responsible for parking slot management, slot status tracking, and inheritance implementation.

Shatha Al Busaidi (24-0143) → Reservation Class

Responsible for reservation creation, reservation tracking, and wrong-slot detection.

Mazayin Al Busaidi (24-0572) → ParkingSession Class, Payment Class & Main Function

Responsible for parking duration calculation, fee processing, penalty calculation, and controlling the program flow through the main menu.

Basaier Al Kharusi (24-0071) → ParkingGateSystem Class

Responsible for system integration, gate operations, parking management, and coordinating all system components.

